

NXP WIT BATCH-IV SYNOPSIS SUBMISSION

Batch Name - NXP WIT BATCH 4: GROUP 2

Name of Students - Arunima Chakraborty, Arushi Amit Shesh, Asmita Bhattacharya, Badavathu Pooja Naik, Dalai Ramya, Ddristi Biswas, Disha Sharma

Project Title - RTL Design of SPI Slave in Verilog- HDL

Guided by – Mr. Sanjay Saini, Mr. Shashikant Sharma, Ms. Trisha

Submission Date – 31st August, 2026

Synopsis:

ABSTRACT

Serial Peripheral Interface (SPI) is a synchronous, full-duplex communication protocol widely used to interface controllers with peripheral devices such as sensors, ADCs, DACs, memory devices, displays, and SD cards. This project focuses on the **RTL design and functional verification of an 8-bit SPI Slave using Verilog HDL**. The design implements serial data reception through MOSI and serial data transmission through MISO, with support for all four standard SPI modes using configurable CPOL and CPHA inputs. An internal system clock is used to process the externally generated SPI signals through synchronization and clock-edge detection logic. The design incorporates transmit and receive shift registers, synchronization registers, a bit counter, and control logic to manage data transfer. A Verilog-based testbench is developed to emulate SPI master behaviour and verify the slave under different SPI modes and data patterns. The design is simulated using **EDA Playground with Icarus Verilog**, and the resulting waveforms are analysed using **EP Wave**. The simulation results demonstrate correct 8-bit data reception and transmission across the supported SPI modes and confirm the generation of the rx_valid signal after a complete data word is received.

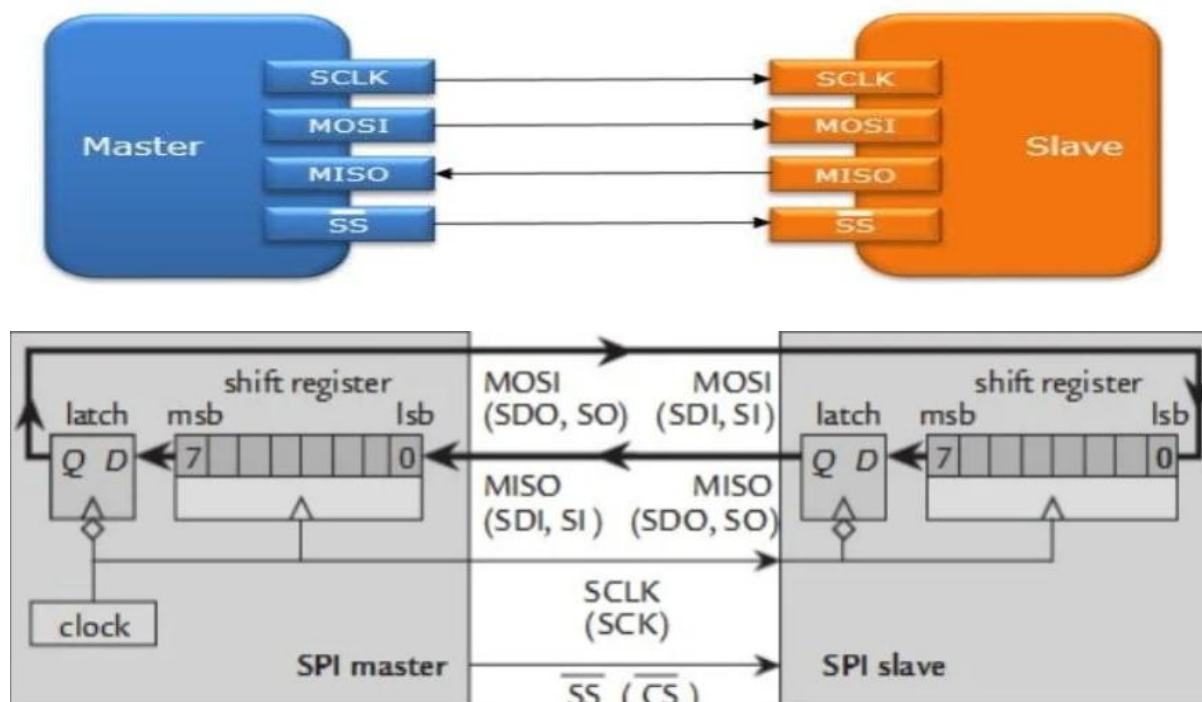
INTRODUCTION

Modern embedded and digital systems often consist of multiple devices that need to exchange information with each other. A microcontroller, processor, FPGA, or other controller may need to communicate with sensors, memories, converters, displays, and other peripheral integrated circuits. To enable this communication, different serial and parallel communication interfaces are used. Serial interfaces are particularly useful because they require fewer physical connections and can provide efficient communication between devices.

Among the commonly used serial interfaces, Serial Peripheral Interface (SPI) is widely adopted for communication between a controller and peripheral devices. SPI was developed as a simple synchronous interface in which the timing of data transfer is controlled by a clock generated by the master. Unlike asynchronous protocols, where the transmitter and receiver must agree on timing parameters, SPI provides an explicit clock signal with the data, making the timing relationship between the communicating devices well defined.

SPI COMMUNICATION ARCHITECTURE

SPI follows a master-slave architecture. The master initiates communication and controls the timing of the transfer, while the slave responds to the commands and clock provided by the master. A system may contain one or multiple slave devices connected to the same master.



SPI uses four main signals for communication between the master and slave:

Signal	Full Name	Direction	Function
SCLK	Serial Clock	Master → Slave	Clock generated by the master. The RTL detects its rising and falling edges to determine when data should be sampled or shifted.
MOSI	Master Out Slave In	Master → Slave	Carries serial data from the master to the slave. In the RTL, MOSI is synchronized and shifted into the receive shift register.
MISO	Master In Slave Out	Slave → Master	Carries serial data from the slave to the master. The transmit shift register provides the data shifted out through MISO.
CS	Chip Select	Master → Slave	Active-low signal used to select the slave and define the SPI transaction. The RTL detects the transition from HIGH to LOW to identify the start of a transaction.

Role of Clock and SPI Modes

The SPI clock (SCLK) is generated by the master and provides the timing reference for data transmission and reception. Since SPI is synchronous, the slave uses SCLK edges to

determine when to sample incoming data and shift outgoing data. The sampling and shifting edges depend on the selected SPI clock mode.

SPI supports four modes, determined by Clock Polarity (CPOL) and Clock Phase (CPHA).

CPOL defines the idle state of SCLK, while **CPHA** determines the clock edge used for data sampling.

SPI MODE	CPOL	CPHA	SHIFT SCK EDGE	CAPTURE SCK EDGE
0	0	0	Falling	Raising
1	0	1	Raising	Falling
2	1	0	Raising	Falling
3	1	1	Falling	Raising

Working of the SPI Interface

When the master initiates communication, it asserts the CS signal LOW to select the slave and generates the SCLK pulses. Data is transferred serially through MOSI, while the selected slave can simultaneously transmit data through MISO, making SPI a full-duplex interface.

The transfer is synchronized with SCLK. For an 8-bit transfer, eight data bits are exchanged, with the sampling and shifting edges determined by the selected CPOL and CPHA mode.

MSB and LSB Data Transfer

SPI commonly transfers data MSB first. For an 8-bit word, the transfer sequence is:

D7 → D6 → D5 → D4 → D3 → D2 → D1 → D0

Shift registers are used to perform the serial-to-parallel and parallel-to-serial conversion. During reception, bits arriving through MOSI are sequentially collected into a receive shift register. During transmission, the parallel transmit data is loaded into a transmit shift register and shifted out bit by bit through MISO.

OBJECTIVES

- To design and implement an 8-bit SPI Slave module at the RTL level using Verilog HDL.
- To implement reliable serial data reception through MOSI and serial data transmission through MISO using shift-register-based logic.
- To support all four standard SPI modes (Mode 0, Mode 1, Mode 2, and Mode 3) using configurable CPOL and CPHA inputs.
- To synchronize the externally generated SCLK, CS, and MOSI signals with the internal system clock and detect the required SCLK transitions.
- To develop Verilog-based testbenches that emulate SPI master behaviour for functional verification.
- To verify the design through simulation and waveform analysis using EDA Playground, Icarus Verilog, and EPWave.

METHODOLOGY

1. RTL Design

The SPI Slave is described at the Register Transfer Level using Verilog HDL. The design uses synchronization registers, clock-edge detection logic, CPOL/CPHA configuration, transmit and receive shift registers, a bit counter, and control logic to manage SPI data transfer.

2. SPI Signal Synchronization

Since SCLK, CS, and MOSI are externally generated SPI signals, they are passed through synchronization registers using the internal system clock. The synchronized SCLK is compared with its previous value to detect rising and falling edges.

3. Data Reception and Transmission

During an active SPI transaction, incoming MOSI data is shifted into the receive shift register. At the same time, the transmit shift register provides outgoing data through MISO. After the required 8 bits have been received, the parallel received data is transferred to rx_data and rx_valid indicates successful reception.

4. SPI Mode Handling

The design determines the appropriate sampling and shifting behaviour according to the selected CPOL and CPHA values. This enables the same RTL module to operate in all four standard SPI modes.

5. Functional Verification

Verilog testbenches are developed to emulate the behavior of an SPI master. The testbench generates the system clock, SCLK, CS, MOSI, CPOL, and CPHA signals and provides test data to the slave.

6. Simulation and Waveform Analysis

The RTL and testbench are simulated using Icarus Verilog on EDA Playground. The generated waveforms are viewed using EPWave to verify SCLK transitions, CS activation, MOSI reception, MISO transmission, received data, and rx_valid.

The design uses separate transmit and receive shift registers to perform serial data transfer.

Reception:

MOSI → RX Shift Register → RX Data

Transmission:

TX Data → TX Shift Register → MISO

The received bits are shifted into the receive register according to the selected SPI sampling edge, while the transmit register shifts the outgoing bits toward MISO. This approach is suitable for the project because SPI fundamentally involves serial shifting of data synchronized with clock edges.

RTL IMPLEMENTATION DETAILS

1. Inputs

Signal	Description
clk	Internal system clock used to synchronize and process SPI signals
rst	Active-high reset
cs	Active-low chip-select signal from the master
sclk	SPI clock generated by the master
mosi	Serial data input from the master
tx_data	8-bit parallel data to be transmitted by the slave
cpol	Selects SPI clock polarity
cpha	Selects SPI clock phase

2. Outputs

Signal	Description
miso	Serial data transmitted from slave to master
rx_data	8-bit parallel data received from MOSI
rx_valid	Indicates that a complete 8-bit data word has been received

MAIN RTL BLOCKS

1. Synchronization Logic

External SCLK, CS, and MOSI signals are synchronized with the internal clk using register stages, allowing safe processing within the system clock domain.

2. SCLK Edge Detection

The synchronized SCLK is compared with its previous state to detect rising and falling edges, which control data sampling and shifting.

3. CPOL/CPHA Logic

The cpol and cpha inputs select one of the four SPI modes and determine the appropriate clock edge for sampling and shifting.

4. Receive Shift Register

rx_shift collects serial data from MOSI. After 8 bits are received, the complete word is transferred to rx_data.

5. Transmit Shift Register

tx_shift stores the transmit data and shifts it sequentially to MISO, with the MSB transmitted first.

6. Bit Counter

bit_count tracks the number of received bits and identifies completion of an 8-bit transfer.

7. Receive Valid Signal

After the eighth bit is received, rx_valid is asserted to indicate that valid data is available in rx_data.

TESTBENCH

The testbench is used to verify the functionality of the SPI Slave without requiring physical hardware.

It emulates the behaviour of an SPI master by generating:

- Internal clk
- Reset
- Chip Select (CS)
- SPI Clock (SCLK)
- MOSI data
- CPOL and CPHA configuration
- Transmit test data

The testbench then observes:

- MISO
- rx_data
- rx_valid
- SCLK and CS timing
- Overall data-transfer behavior

Separate testbench configurations are used to verify the different SPI modes.

```
testbench.sv
41
42 // Mode 0
43 cpol = 0;
44 cpha = 0;
45 tx_data = $random;
46 $display("TX DATA = %b", tx_data);
47 #20;
48 rst = 0;
49 #20;
50 cs = 0;
51 // Send Data
52 mosi = tx_data[7]; #20; sclk = 1; #20; sclk = 0;
53 mosi = tx_data[6]; #20; sclk = 1; #20; sclk = 0;
54 mosi = tx_data[5]; #20; sclk = 1; #20; sclk = 0;
55 mosi = tx_data[4]; #20; sclk = 1; #20; sclk = 0;
56 mosi = tx_data[3]; #20; sclk = 1; #20; sclk = 0;
57 mosi = tx_data[2]; #20; sclk = 1; #20; sclk = 0;
58 mosi = tx_data[1]; #20; sclk = 1; #20; sclk = 0;
59 mosi = tx_data[0]; #20; sclk = 1; #20; sclk = 0;
60 // End SPI
61 #20;
62 cs = 1;
63 #100;
64
65
```

Fig:-SPI Mode 0 TB

```
testbench.sv
41
42 // Mode 1
43 cpol = 1;
44 cpha = 1;
45 tx_data = $random;
46 $display("TX DATA = %b", tx_data);
47 #20;
48 rst = 0;
49 #20;
50 cs = 0;
51 // Send Data
52 #20; sclk = 1; mosi = tx_data[7]; #20; sclk = 0;
53 #20; sclk = 1; mosi = tx_data[6]; #20; sclk = 0;
54 #20; sclk = 1; mosi = tx_data[5]; #20; sclk = 0;
55 #20; sclk = 1; mosi = tx_data[4]; #20; sclk = 0;
56 #20; sclk = 1; mosi = tx_data[3]; #20; sclk = 0;
57 #20; sclk = 1; mosi = tx_data[2]; #20; sclk = 0;
58 #20; sclk = 1; mosi = tx_data[1]; #20; sclk = 0;
59 #20; sclk = 1; mosi = tx_data[0]; #20; sclk = 0;
60 // End SPI
61 #20;
62
63
```

Fig:-SPI Mode 1 TB

```
testbench.sv
40      mosi = 0;
41
42      // Mode 2
43      cpol = 1;
44      cpha = 0;
45      tx_data = $random;
46      $display("TX DATA = %b", tx_data);
47      #20;
48      rst = 0;
49      #20;
50      cs = 0;
51      // Send Data
52      mosi = tx_data[7];#20;sclk = 0; #20; sclk = 1;
53      mosi = tx_data[6]; #20;sclk = 0; #20; sclk = 1;
54      mosi = tx_data[5];#20;sclk = 0; #20; sclk = 1;
55      mosi = tx_data[4];#20;sclk = 0; #20; sclk = 1;
56      mosi = tx_data[3];#20;sclk = 0; #20; sclk = 1;
57      mosi = tx_data[2];#20;sclk = 0; #20; sclk = 1;
58      mosi = tx_data[1];#20;sclk = 0; #20; sclk = 1;
59      mosi = tx_data[0]; #20;sclk = 0; #20; sclk = 1;
60      // End SPI
61      #20;
62      cs = 1;
63
64      #100;
```

Fig:SPI Mode 2 TB

```
testbench.sv
40      mosi = 0;
41
42      // Mode 3
43      cpol = 1;
44      cpha = 1;
45      tx_data = 8'b10101010;
46      $display("TX DATA = %b", tx_data);
47      #20;
48      rst = 0;
49      #20;
50      cs = 0;
51      // Send Data
52      mosi = tx_data[7];#20;sclk = 0; #20; sclk = 1; #20;
53      mosi = tx_data[6]; #20;sclk = 0; #20; sclk = 1; #20;
54      mosi = tx_data[5];#20;sclk = 0; #20; sclk = 1; #20;
55      mosi = tx_data[4];#20;sclk = 0; #20; sclk = 1; #20;
56      mosi = tx_data[3];#20;sclk = 0; #20; sclk = 1; #20;
57      mosi = tx_data[2];#20;sclk = 0; #20; sclk = 1; #20;
58      mosi = tx_data[1];#20;sclk = 0; #20; sclk = 1; #20;
59      mosi = tx_data[0]; #20;sclk = 0; #20; sclk = 1;
60      // End SPI
61      #20;
62      cs = 1;
63
```

Fig:SPI Mode 3 TB

VERIFICATION

The design is verified by performing SPI transactions and comparing the transmitted and received data.

The verification checks:

- Correct activation of the slave using CS.
- Correct synchronization of external SPI signals.
- Correct detection of SCLK rising and falling edges.
- Correct selection of sampling and shifting edges according to CPOL/CPHA.
- Correct reception of 8-bit MOSI data.
- Correct transmission of data through MISO.
- Correct generation of rx_valid after receiving a complete byte.
- Correct operation across Modes 0, 1, 2, and 3.
- Correct waveform relationship between SCLK, MOSI, MISO, CS, rx_data, and rx_valid.

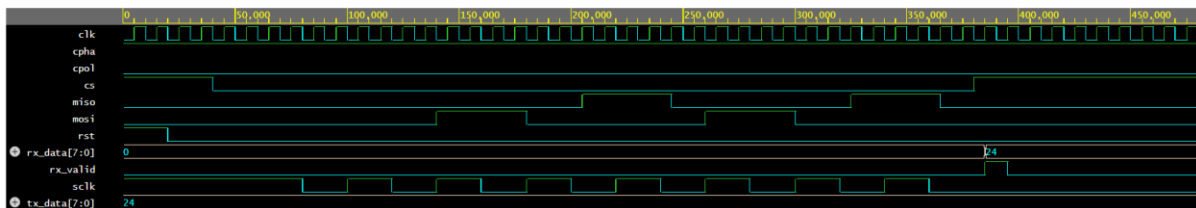


Fig: SPI Mode 1 Simulation Waveform

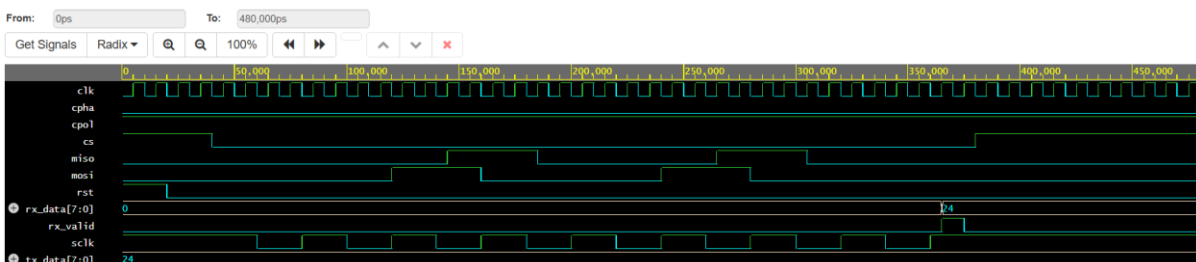


Fig: SPI Mode 2 Simulation Waveform

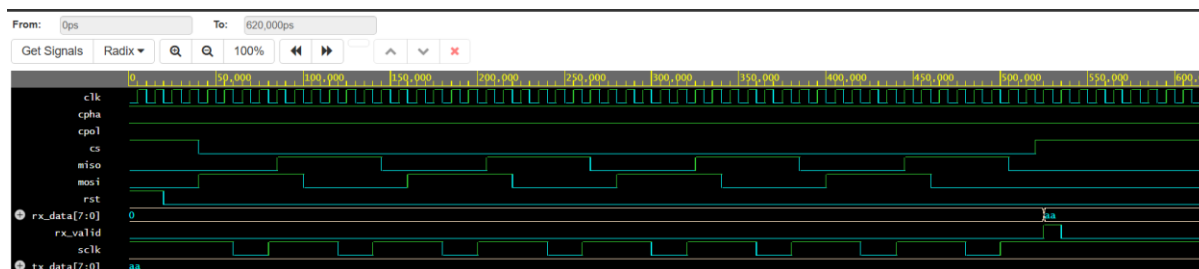


Fig: SPI Mode 3 Simulation Waveform

APPLICATIONS

Device / IC	Category	SPI Mode	Max Clock	Notes
SD / microSD Card	Storage	Mode 0	25 MHz	SPI mode is a compatibility fallback. SDIO is faster for production. CS=DAT3, CMD=MOSI, DAT0=MISO.
W25Q128 NOR Flash	Storage	Mode 0 / Mode 3	104 MHz (STD), 133 MHz (QPI)	Winbond W25Q series — industry-standard SPI NOR. Supports Dual and Quad SPI for 2x / 4x throughput.
SSD1306 OLED	Display	Mode 0	10 MHz	128x64 monochrome OLED. Uses extra D/C (Data/Command) pin alongside SPI. Also available in I2C.
ILI9341 TFT LCD	Display	Mode 0 / Mode 3	42 MHz (write)	240x320 colour TFT. SPI pixel writes common on microcontrollers. Parallel bus used for higher refresh rates.
ADS1115 ADC	ADC	Mode 1	400 kHz (I2C alt); SPI-like via custom clocking	16-bit, 860 SPS. Typically accessed over I2C, but concept applies to similar SPI ADCs like MCP3208.
MCP4921 DAC	DAC	Mode 0 / Mode 3	20 MHz	12-bit single-channel DAC from Microchip. 16-bit SPI frame: 4 config bits + 12 data bits. LDAC pin latches output.
nRF24L01+ Radio	Wireless	Mode 0	10 MHz	2.4 GHz ISM band transceiver. SPI used for register config and FIFO access. IRQ pin signals events.
ENC28J60 Ethernet	Networking	Mode 0	20 MHz	10BASE-T MAC/PHY with SPI interface. Widely used in embedded Ethernet. Buffer managed via SPI register commands.

CONCLUSION

The project successfully designed and tested an 8-bit SPI Slave using Verilog HDL. The slave can receive data through MOSI and transmit data through MISO while supporting all four SPI modes. The simulation results confirm the correct working of the design and data transfer.

REFERENCES

<https://ecrionix.org/serial-protocols/spi/>

<https://nerdyelectronics.com/spi-protocol-deep-dive/>